Lecture 15 : 25th Feb 2026 :

The Dual func^c

$$q(\mu) = \inf_{w,b} \left\{ \frac{1}{2} w^T w + \sum_{i=1}^{n} \mu_i \{1 - y_i (w^T x_i + b)\} \right\}$$

observe the term $\sum \mu_i y_i b$, by the choice of μ_i & b it can drive inf to $-\infty$

$\therefore$ solve the Constraint version by forcing $\sum_i \mu_i y_i = 0$

To solve the problem, substitute

$$w^* = \sum_{i \in S} \mu_i y_i x_i$$

$$b^* = \quad \text{H.W}$$

now by substituting the optimal values of μ^* & b^* we get the optimal value for dual func^c

$$\therefore q(\mu) = \sum_{i=1}^{n} \mu_i - \frac{1}{2} \sum_i \sum_j \mu_i y_i \mu_j y_j x_i^T x_j$$

In the dual problem, data x, only appears as inner products

Now the dual optimization problem:

$$\max \sum_i \mu_i - \frac{1}{2} \sum_i \sum_j \mu_i \mu_j y_i y_j x_i^T x_j$$

$$s.t \quad \mu_i \geq 0$$

$$\sum_i y_i \mu_i = 0$$

The above problem is quadratic in μ_i, w.r.t the linear constraints : qp problem with linear constraint

* One of the standard qp solvers

SMO : Sequential Min Optimizers (Lib SVM)

Once μ_i^* are obtained find

$$w^* = \sum y_i \mu_i^* x_i$$

III by get b^*

* Support vectors are those data points for which $\mu_i^* > 0$

* From one of the KKT conditions,

$$\mu_i^* \left[1 - y_i (w^T x_i + b) \right] = 0$$

for Support Vectors.

$$y_i (w^T x_i + b) = 1$$

⇒ These data points are on the Support Vectors. Since we are assuming lin Sep. ⇒ these are the farthest data points on the class near decision boundary.

SVM for Not Linear Seperable Case:

⇒ $\nexists\ w$ s.t $y_i (w^T x_i + b) > 1 \quad \forall\ i$

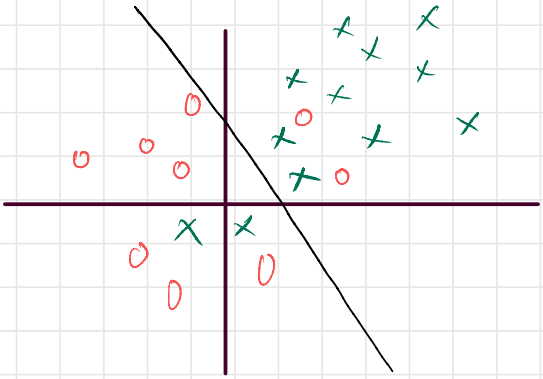
→ Introduce another additional variable into to optimization that quantifies the extent of mis-classification.

→ The original problem

$$\min_{w} \frac{1}{2} w^T w$$

$$\text{s.t} \quad y_i (w^T x_i + b) > 1$$

→ Now the modification is



mistake has been accounted by ξ_i

Slack Variable.

→ Un Constrained Slack may lead to a trivial Solution. (for example $w=0$)

∴ we should have Some Constraint

and now the optimization looks like.

$$\min_{w,\xi} \quad \frac{1}{2} w^T w + C \sum_{i=1}^{n} \xi_i$$

$$s.t \quad y_i (w^T x_i + b) \geq 1 - \xi_i \qquad \forall i$$

$$\xi_i \geq 0$$

+Slack with ξ_i 0-1, Correctly Classified with the margin

* Slack with $\xi_i > 1$, misclassified.

$$L(w, b, \mu, \lambda, \xi) = \frac{1}{2} w^T w + C \sum_{i=1} \xi_i + \sum_i \mu_i \{ 1 - \xi_i - y_i (w^T x_i + b) \} - \sum \lambda_i \xi_i$$

Now KKT for the above problem:

i) $\nabla_w L = 0 \Rightarrow w^* = \sum_i \mu_i^* y_i x_i$

ii) $\nabla_b L = 0 \Rightarrow \sum_i \mu_i^* y_i = 0$

iii) $\nabla_\xi L = 0 \Rightarrow \mu_i^* + \lambda_i^* = C$

iv) $\mu_i (1 - \xi_i - y_i (w^T x_i + b)) = 0$ &

$\lambda_i \xi_i = 0 \quad \forall i$

Formulating the Dual problem:

$$q(\mu, \lambda) = \inf_{w, b, \xi} L(w, b, \xi, \mu, \lambda)$$

Observe that "L" has a term.

$$\sum (C - \mu_i - \lambda_i) \xi_i$$

we we do not put a constraint, it can take the inf problem to trivial soln of $-\infty$

by $\mu_i + \lambda_i = C$

* Dual problem :

$$\max_{\mu, \lambda} \sum_i \mu_i - \frac{1}{2} \sum_i \sum_j y_i y_j \mu_i \mu_j x_i^T x_j$$

s.t $\mu_i \geq 0$

$\lambda_i \geq 0$ and

$C = \mu_i + \lambda_i$

To take out "λ" from optimization problem.

modify the Constraint as $0 \leq \mu_i \leq C$

$$\max_{\mu} \sum_i \mu_i - \frac{1}{2} \sum_i \sum_j y_i y_j \mu_i \mu_j x_i^T x_j$$

$$s.t \quad \mu_i \geq 0, \quad 0 \leq \mu_i \leq C$$

SVM with Kernel Func^c :

→ why does the decision boundry needs to be a Line?

→ Define a func^c : $\phi : \mathbb{R}^d \to \mathbb{R}^{d'}$

typically $d' >> d$

* with this, the dual func^c to be solved will be

$$\max_{\mu} \sum \mu_i - \frac{1}{2} \sum_i \sum_j \mu_i \mu_j y_i y_j \phi(x_i)^T \phi(x_j)$$

$$s.t \quad 0 \leq \mu_i \leq C.$$

* Suppose $\exists$ a func^c k :

$$k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R} \quad s.t$$

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad \forall \ i,j$$

↳ kernel func^c

Now the SVM dual problem will be

$$\sum \lambda_i - \frac{1}{2} \sum \sum \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

$$s.t \quad 0 \leq \lambda_i \leq C$$

if we know the kernel fun^c, we do not need to know even the projections also.

Mercer's Theorem:

Suppose $\bar{K}$ is an $n \times n$ matrix with

$$\bar{K}_{ij} = K(x_i, x_j)$$

if $\bar{K}$ is PSD $\forall$ D, then K is a valid kernel (one that corresponds to proj ϕ)

* No need to worry about projection fun^c, get the kernel fun^c, check is it PSD, then the problem solve.

Examples of Valid Kernels:

1) poly Kernel :

$$K_p(x_1, x_2) = (1 + x_1, x_2)^p$$

2) Sigmoid Kernel :

$$K_s(x_1, x_2) = \tanh(a x_1^T x_2 + b)$$

The final discriminant func is

$$g(\phi(x)) = w^T \phi(x)$$

$$= \left[\sum_s \lambda_i^* y_i \phi(x_i) \right]^T \phi(x)$$

$$= \sum_s \lambda_i \cdot y_i \tanh(a x_i^T x + b)$$

3) Gaussian Kernel.

$$K_G(x_1, x_2) = \exp\left\{ \frac{-\|x_1 - x_2\|^2}{\sigma^2} \right\}$$

s.t Kernel that we use.

SVM as Reg-ERM.

→ $\min_{w,b,\xi} \quad \frac{1}{2} w^T w + C \sum \xi_i$

$$\text{s.t} \quad y_i (w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

→ Given any w & b, ξ_i has to satisfy.

$$\xi_i \geq \max\left(0, \; 1 - y_i (w^T x_i + b)\right)$$

now:

$$\min_{w,b} \quad \frac{1}{2} w^T w + C \sum_{i=1}^{\hat{}} \max\left(0, \; 1 - y_i (w^T x_i + b)\right)$$

$$\Omega(w) + C \sum L(h(x_i), y_i)$$

$$L(h(x), y) = \underbrace{\max\left(0, \; 1 - y \, h(x_i)\right)}$$

hinge Loss.

Lecture 16 : 2nd March 2026:

Neural Networks :

$x \in \mathbb{R}^d, \quad y \in \mathbb{R}$

$h_\theta(x) = \sigma\left(w_2^T \sigma(w_1^T x)\right)$: Neural N/w

Composite fun^n comprising alternating linear and non-linear mappings.

Sign $(w^T x)$: perceptron.

Perceptron Learning Algo : Do in tutorial

if data is linearly seperable, it converges in finite steps.

Multi Layer Perceptron :

$x \in \mathbb{R}^d, \quad y \in \mathbb{R}^k$

$w_3^T \sigma\left[w_2^T \left[\sigma(w_1^T x) \right] \right]$

$w_1 \in \mathbb{R}^{L_1 \times d}$

$w_3 \in \mathbb{R}^{L_2 \times k}$ $\quad$ $w_2 \in \mathbb{R}^{L_2 \times L_1}$

$\theta = [w_1, w_2, w_3]$

$\sigma(\ )$: element wise non-linearity

eg : $\sigma(t) = \frac{1}{1+e^{-t}}$

Universal Approximation Theorem:

Let X be a compact subset of $\mathbb{R}^n$ &

$C(X)$ denote the space of all continuous fun^c :

$f : X \to \mathbb{R}, \quad x \in \mathbb{R}^d$

Let $\sigma : \mathbb{R} \to \mathbb{R}$, s.t its bounded, continuous &

$\lim_{t \to \infty} \sigma(t) = 1$ & $\lim_{t \to -\infty} \sigma(t) = 0$

Then for any target fun^{c} $f \in C(x)$

& $\epsilon > 0$, $\exists$ an MLP with single hidden Layer with N neurons s.t

$$h(x) = \sum_{i=1}^{n} w_2 \, \sigma(w_1^T x + b)$$

Satisfies

$$\sup_{x \in X} | h(x) - f(x) | < \epsilon$$

where w_2 & w_1 are parameters.

$$w_1 \in \mathbb{R}^{N \times d} \qquad x \in \mathbb{R}^{d \times 1}$$

$$w_2 \in \mathbb{R}^{1 \times N}$$

Shallow / Depth ⟶

wide / Narrow ↓

→ now a days we use, Deep Narrow n/w

ERM on Neural Networks:

$X \in \mathbb{R}^d, \quad Y \in \mathbb{R}$

$$D = \{ (x_i, y_i) \}_{i=1}^{n}$$

$$h_\theta(x) = w_2 \, \sigma(w_1 x)$$

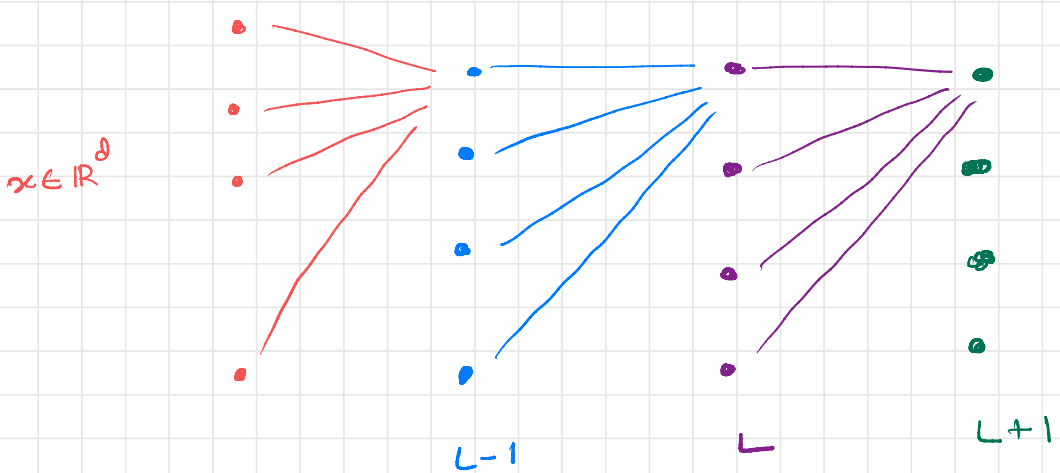
$$\theta^* = \operatorname*{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(h_\theta(x_i), y_i)$$

for NNs, we use grad descent

$$\theta^{t+1} \leftarrow \theta^t - \alpha \nabla_\theta \hat{R}(\theta)$$

how to compute the gradients $\nabla_\theta \hat{R}(\theta)$ for NNs ?

Soln: Chain Rule. : Error Back propagation.



Notations: L : Current Layer

j : index of the neuron in L^{th} layer.

k : index of neuron in $(L-1)^{th}$ layer

i : index of a neuron in $(L+1)^{th}$ layer

$w^L_{j,k}$: weight connecting neuron k to neuron j

b^L_j : bias of neuron j in layer L.

Define :

$$z^L_j = \sum_k w^L_{jk} a^{L-1}_k + b^L_j$$

$$a^L_j = \sigma(z^L_j)$$

z^L_j : pre activation output of neuron j in layer L

a^L_j : Activation (a) output of j^{th} neuron in L^{th} layer.

To compute the Gradients:

Suppose $\hat{R}$ denotes the emp risk.

we want $\frac{\partial \hat{R}}{\partial w_{jk}^{l}}$

Define the Error for a single neuron in layer "l" as follows.

$$\delta_j^l = \frac{\partial}{\partial z_j^l} \hat{R}$$

To compute error for the final L^{th} layer.

$$\delta_j^L = \frac{\partial}{\partial z_j^L} \hat{R}$$

$$= \frac{\partial}{\partial a_j^L} \hat{R} \cdot \frac{\partial}{\partial z_j^L} a_j^L$$

Note: $h_\theta(x)_j = a_j^L$

$$= \sigma(z_j^L)$$

$$\frac{\partial}{\partial z_j^L} a_j^L = \sigma'(z_j^L)$$

$$\frac{\partial}{\partial a_j^L} \hat{R} = \frac{\partial}{\partial a_j^L} \left[L(a_j^L, y_j) \right]$$

Can be computed.

To compute δ_j^L :

$$\delta_j^L = \frac{\partial}{\partial z_j^L} \hat{R}$$

$$= \sum_i \frac{\partial}{\partial z_j^{L+1}} \hat{R} \quad \frac{\partial}{\partial z_j^L} z_i^{L+1}$$

$$= \sum_i \delta_i^{L+1} \; (\;)$$

Consider $\frac{\partial}{\partial z_j^L} z_i^{L+1}$

$$z_i^{L+1} = \sum_j w_{ij}^{L+1} a_j^L + b_i^{L+1}$$

$$= \sum_j w_{ij}^{l+1} \sigma(z_j^l) + b_i^{l+1}$$

$$\frac{\partial}{\partial z_j^l} z_i^{l+1} = w_{ij}^{l+1} \sigma'(z_j^l)$$

Once we substitute this,

$$\delta_j^l = \sum_i \delta_i^{l+1} \cdot w_{ij}^{l+1} \sigma'(z_j^l)$$

Error in Layer l, gets propogated backwards via the weights & hence the name.

its a linear comb^n of error at $l+1$ layer & the weights connecting l & $l+1$ layer.

To compute $\frac{\partial}{\partial w_{jk}^l} \hat{R}$

$$\frac{\partial}{\partial w_{jk}^l} \hat{R} = \frac{\partial}{\partial z_j^l} \hat{R} \; \frac{\partial}{\partial w_{jk}^l} z_j^l$$

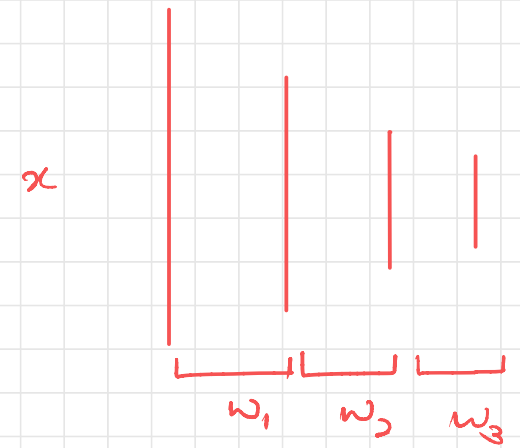
Fill in the Steps.

$= \delta_j^L \; a_k^{L-1}$

Computed Via back prop

Computed via forward prop

Forward prop to compute a^L



Back prop to obtain $\frac{\partial}{\partial w} \hat{R}$

To get one gradient step,

1) Forward pass

2) Backward pass.